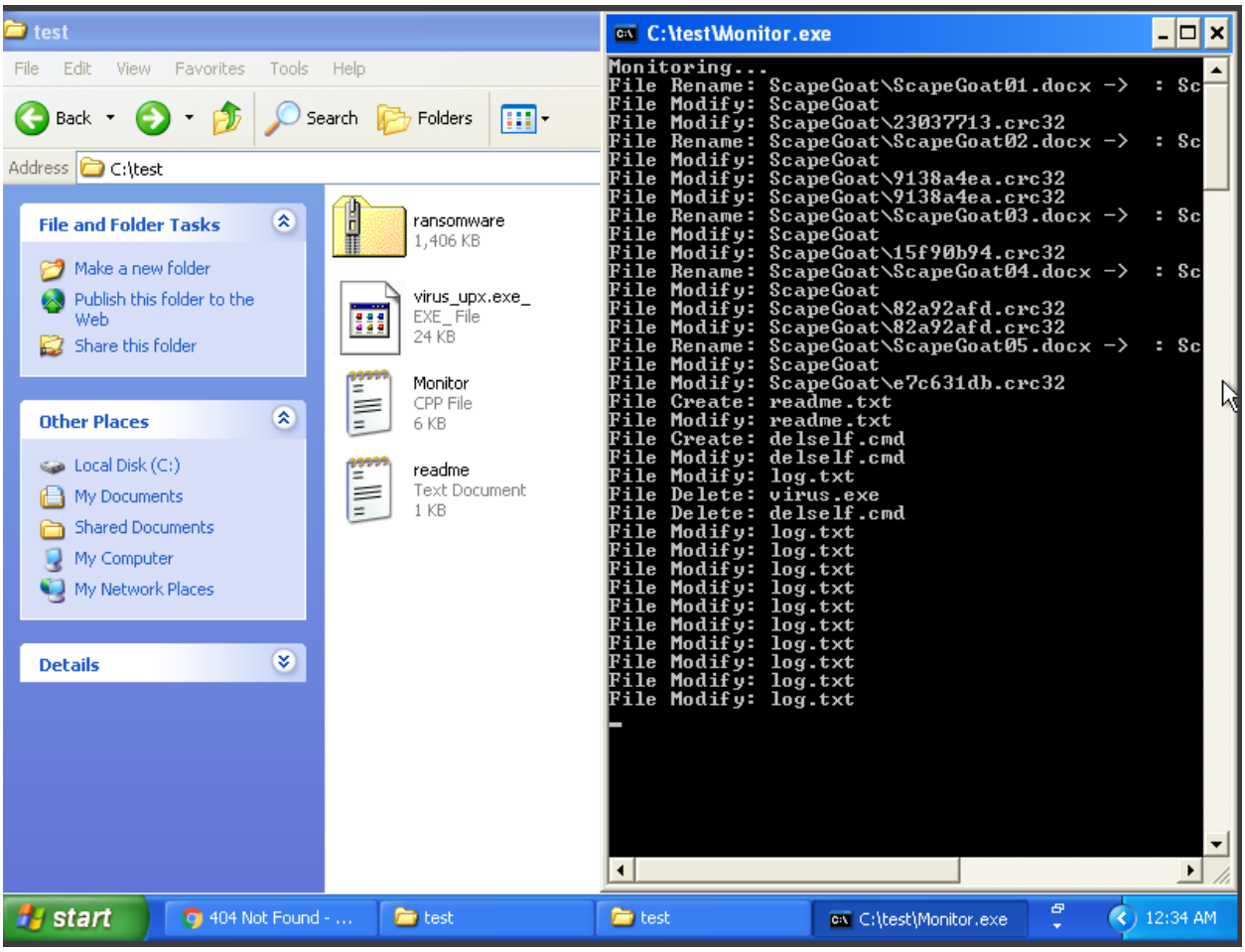
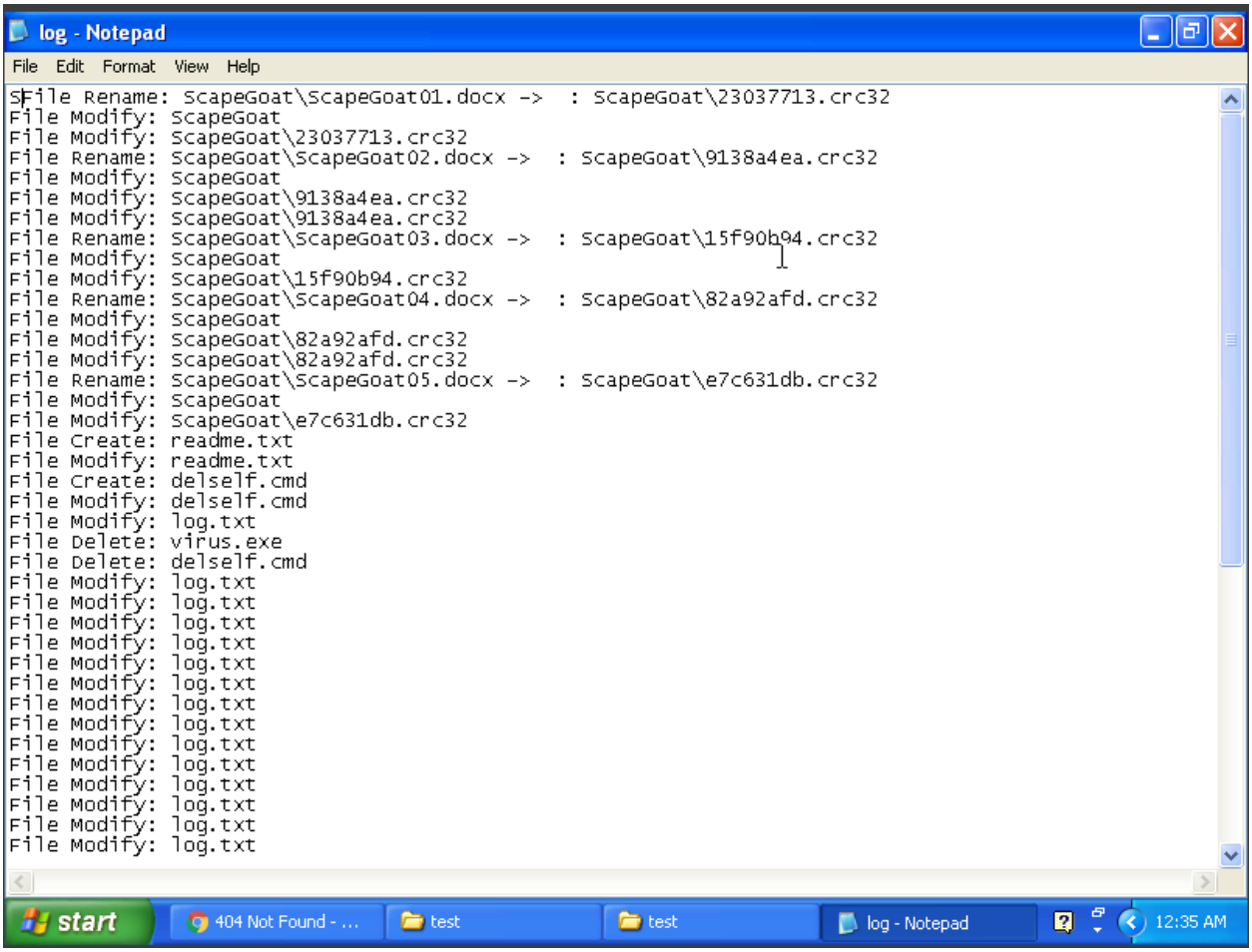
Lab 4 Report by Alex Cooper

In the lab4 assignment the goal is to create a python file that will detect a virus based on a set of 3 rules. This file is meant to mimic a “ransomware” virus, a type of virus that once installed generally is followed some sort of exploitation to for the attackers benefit or the victim’s harm. If the demands are met the virus will then remove itself from the machine.

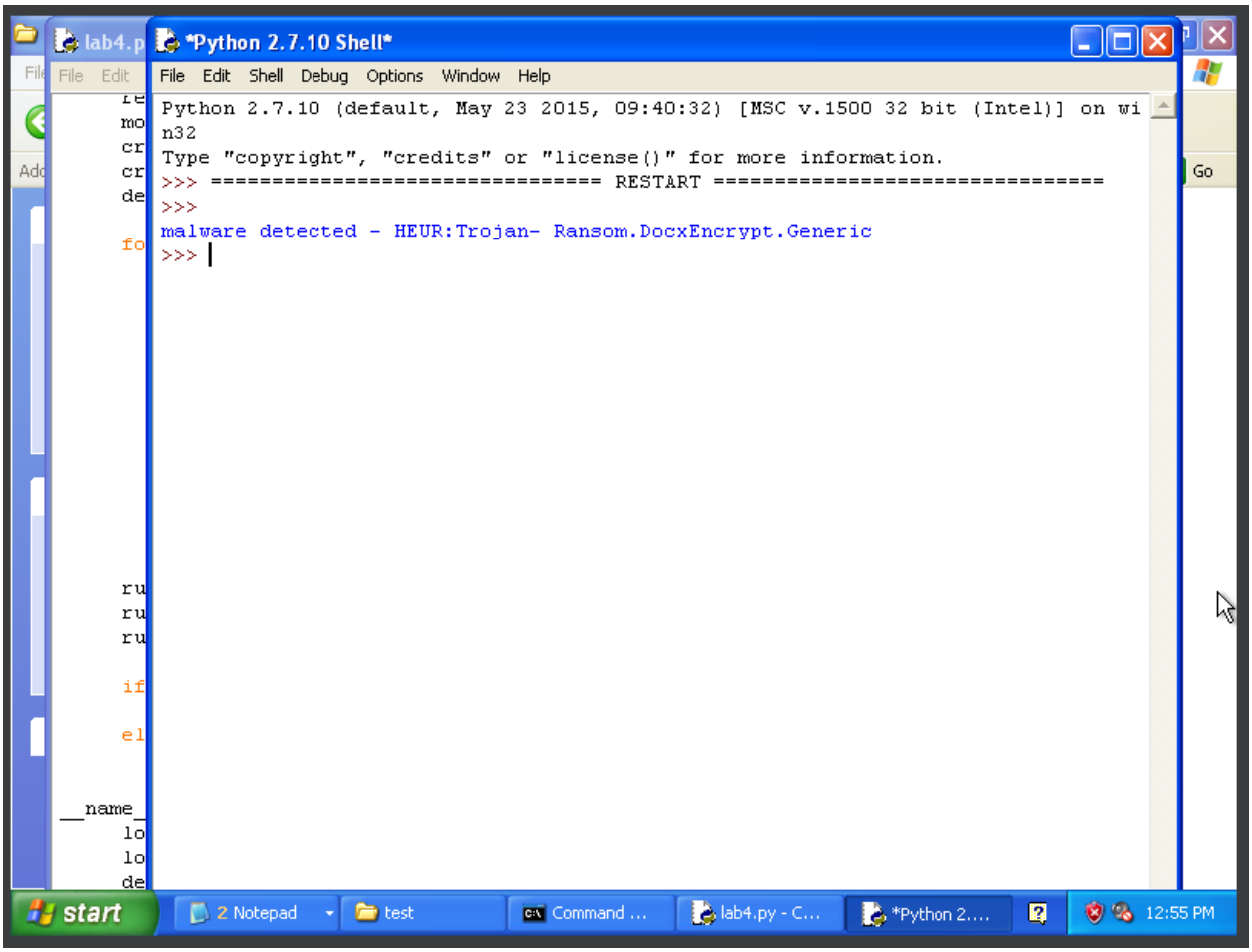
The lab requires the usage of a Windows XP environment to accurately complete the assignment. For this example, I used the courses provided image and loaded it up using the Virtual Box app to create the Virtual Machine. In the screenshot below you will see this environment after the completion of all the initial startup steps. You can see the “address” ir directory is “C:\test” as per instructions. You can also see the zip file has been moved here which allows for the extraction of the lab executables. On the other half you can see the running of monitor.exe after the virus.exe file is run. This same information is written to the logs. If you do not exit the monitor, it will indefinitely loop with “File Modify: log.txt” which is why there are multiple instanced of that toward the bottom of the monitor.



At this point I accessed the logs.txt file to verify that the information was being written. After confirming this I closed the monitoring program.



We can use the above screenshot to take a peek at verifying if the file is a virus or not. In this case we can visually inspect and see the logs indicate multiple .docx files being renamed, multiple files being modified, and an instance of a self-delete. Below is the determination of the code written to detect whether the file is a virus. I downloaded python to the virtual machine and used the built-in environment to run the code, however, wrote the code in notepad and made it a .py file.



Below are the code snippets for the lab. Unfortunately, despite my best efforts, I consistently get errors indicating that the clipboard will not carry over outside of the VM (I did change the settings and reset the VM but had no luck), so screenshots of the snippets are below. The code starts by reading the logs, storing each line to a list and returning this new log list. Next, we do the actual detection using the log list as the input. We begin with local variable initialization then, move right into the loop. First, we check what rules we are checking for. Then since the rules specifically state the renames must occur in the ScapeGoat folder, we check whether the ScapeGoat file path is in the file being renamed and if it's a .docx. If it is then the count goes up one. A similar process is done for the modified files except any file extension can increase the count here. Finally, the create delete has some additional logic. First, we use a loop to separate the newly created files into a list, and the newly deleted files into a separate list. From here we use a loop on the created list to see if an indexed element is also in the deleted list. Per instructions it only counts if the file was created and deleted. This does create a nested loop which generally is not best practice, however, it was the way I knew how to do it. By this point we have the rules logically built so we can check whether these rules are being violated and print the

appropriate output. In this instance I hardcoded the logs.txt and placed this .py file in the same directory for a simpler demonstration of the lab. Though with one line change it could accept any file path input. From there the two functions are ran.